



JavaScript Fundamentals, DOM Manipulation & Introduction to Node.js

An instructor-led training programme for QA Engineers, Automation Testers, and QE/SDET aspirants. This course builds a practical foundation in JavaScript scripting, browser-level DOM interaction, and Node.js runtime essentials — all directly applicable to modern test automation.

INSTRUCTOR-LED TRAINING

BEGINNER TO INTERMEDIATE

Course Roadmap

What We Will Cover Today

01

JavaScript Introduction

Language basics, role in web apps, and QA relevance

02

JS Fundamentals

Variables, data types, operators, conditionals, and loops

03

Functions & Scope

Defining, calling, and structuring reusable logic

04

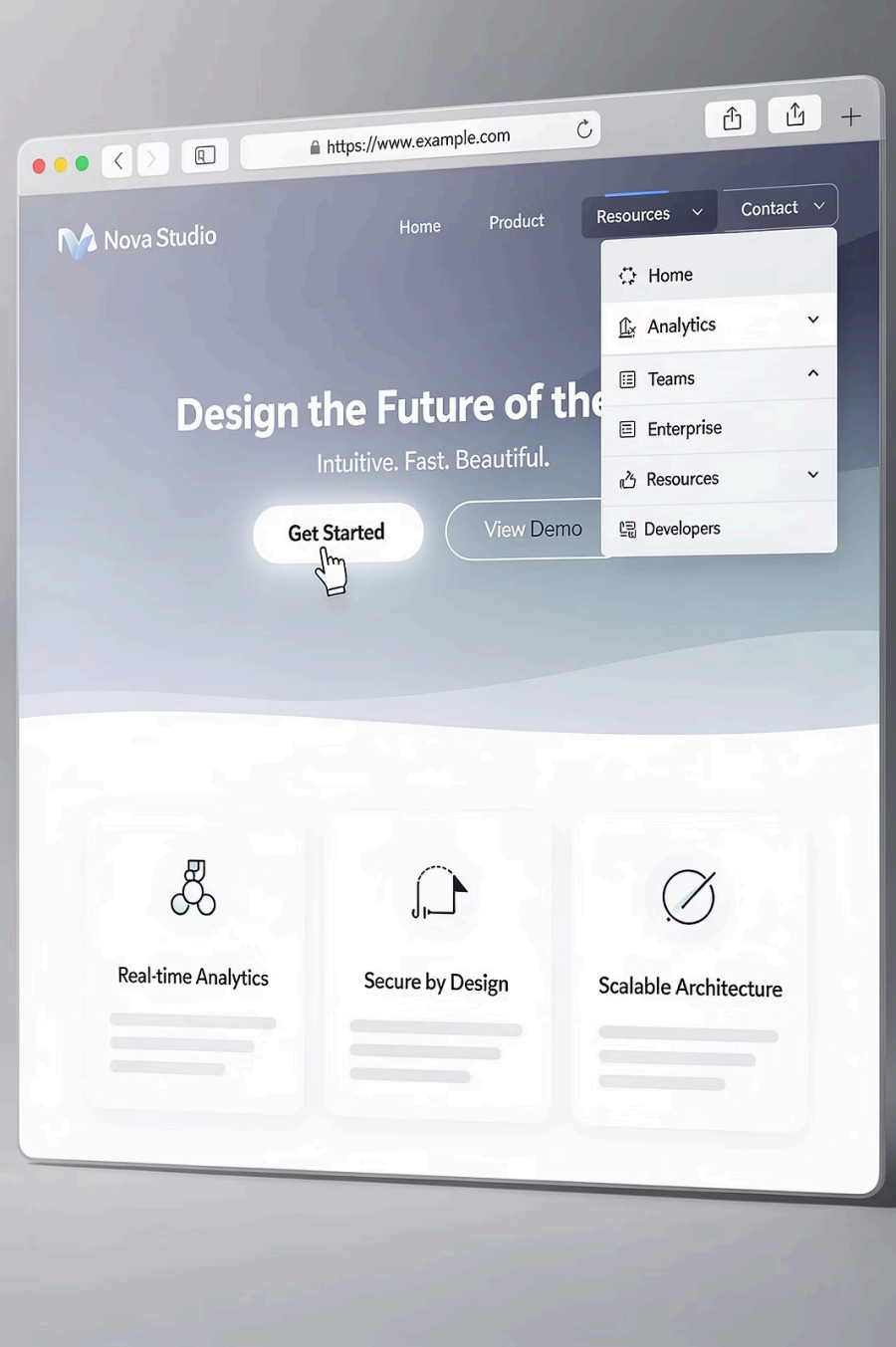
DOM & Manipulation

Accessing and modifying page elements programmatically

05

Events & Node.js

Event handling, Node runtime, modules, and automation tooling



SECTION 1

Introduction to JavaScript

What is JavaScript?

The Definition

JavaScript is a lightweight, interpreted, high-level programming language primarily designed to make web pages interactive. It runs directly in the browser without any compilation step, making it accessible and fast to prototype with.

It is the only programming language natively supported by all major browsers — Chrome, Firefox, Safari, and Edge.

Key Characteristics

→ Client-Side Scripting

Executes in the user's browser, manipulating the page without server round-trips

→ Event-Driven

Code responds to user actions like clicks, keypresses, and form submissions

→ Dynamic Typing

Variables can hold any type; no need to declare data types explicitly

Role of JavaScript in Web Applications

JavaScript is the engine that powers the interactive layer of every modern web application. It works alongside HTML (structure) and CSS (style) to create fully dynamic, responsive user experiences.



Form Validation

Validates user input in real time — checking email formats, required fields, and password strength before data is submitted to the server.



Button Click Events

Responds to user interactions instantly. Clicking a button can toggle menus, submit data asynchronously, or update content on screen.



Dynamic Page Updates

Fetches data from APIs and updates page content without requiring a full page reload — the foundation of Single Page Applications (SPAs).

JavaScript in QA Automation

JavaScript is no longer just for front-end developers. QA engineers use it extensively across the automation stack — from browser scripting to API testing and test framework configuration.

Browser Scripts

Execute JavaScript directly in browser DevTools or via WebDriver to manipulate elements during automated test execution

API Testing Tools

Postman pre/post scripts, Supertest, and Axios are all JavaScript-based tools used for REST API validation

Test Frameworks

Playwright, Cypress, WebDriverIO, and Jest are widely adopted JS-based frameworks for E2E and unit testing

socket.org echo

Baldassari <james@mabl.com>

Baldassari <james@mabl.com>

IS (1)

CHANGE HISTORY



Edge



Safari (WebKit)

Section 1 — Knowledge Check

Quick Quiz: JavaScript Basics

Question 1

Which of the following best describes JavaScript?

- A. A compiled, statically typed language used for server-side programming only
- B. An interpreted, event-driven scripting language that runs natively in browsers
- C. A stylesheet language used for designing web pages
- D. A database query language for web applications

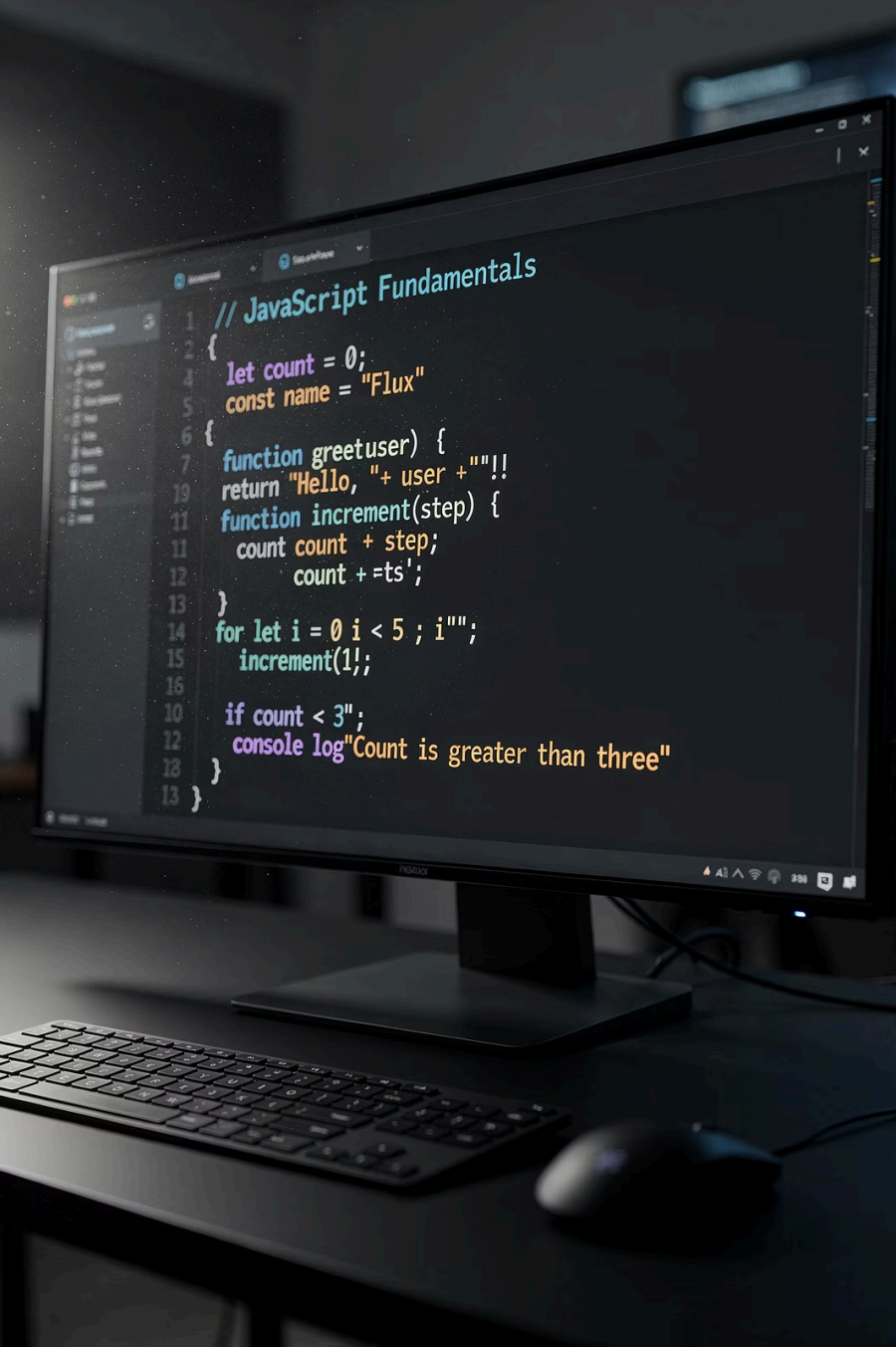
Answer: B — JavaScript is interpreted and event-driven, running natively in all major browsers.

Question 2

Which of the following is a common QA use case for JavaScript?

- A. Designing HTML page layouts
- B. Writing automated browser tests using Playwright or Cypress
- C. Configuring relational databases
- D. Compiling Java source code

Answer: B — Playwright and Cypress are JavaScript-based frameworks widely used in QA automation.



```
// JavaScript Fundamentals
1 {
2   let count = 0;
3   const name = "Flux"
4 }
5 {
6   function greetuser) {
7     return "Hello, "+ user + "!!"
8   }
9   function increment(step) {
10    count count + step;
11    count +=ts';
12  }
13  for let i = 0 i < 5 ; i"";
14    increment(1);
15
16  if count < 3";
17    console log"Count is greater than three"
18  }
19 }
```

SECTION 2

JavaScript Fundamentals

Variables: var, let, and const

Variables are containers for storing data values. JavaScript provides three keywords for declaring variables, each with different scoping rules and mutation behaviour. Choosing the right keyword matters for writing predictable, bug-free scripts.

var

Function-scoped. Can be re-declared and updated. Hoisted to the top of its scope.
Avoid in modern code — its loose scoping causes subtle bugs.

```
var name = "Alice";  
var name = "Bob"; // OK
```

let

Block-scoped. Can be updated but **not re-declared** in the same scope. Preferred for variables that will change value.

```
let count = 0;  
count = 1; // OK
```

const

Block-scoped. Cannot be updated or re-declared. Use for values that should remain **constant** throughout execution.

```
const PI = 3.14;  
// PI = 3 → Error!
```

📌 **Trainer Note:** Advise learners to default to `const`, use `let` when reassignment is needed, and avoid `var` entirely in new scripts.

Data Types in JavaScript

JavaScript is a dynamically typed language — the same variable can hold different types of data at different times. Understanding data types is essential for writing correct conditional checks and assertions in test scripts.

1

String

Text values enclosed in quotes

```
let url = "https://app.com";
```

2

Number

Integers and decimals

```
let timeout = 5000;
```

3

Boolean

True or false logical values

```
let isLoggedIn = true;
```

4

Array

Ordered list of values

```
let browsers = ["Chrome","Firefox"];
```

5

Object

Key-value pairs for structured data

```
let user = { name: "Alice", role: "QA" };
```

Operators in JavaScript

Arithmetic & Assignment

```
let a = 10, b = 3;  
console.log(a + b); // 13  
console.log(a - b); // 7  
console.log(a * b); // 30  
console.log(a / b); // 3.33  
console.log(a % b); // 1 (modulus)  
  
let x = 5;  
x += 2; // x = 7  
x *= 3; // x = 21
```

Comparison & Logical

```
// Comparison (returns boolean)  
console.log(5 == "5"); // true (loose)  
console.log(5 === "5"); // false (strict)  
console.log(10 > 3); // true  
console.log(10 !== 10); // false  
  
// Logical operators  
let a = true, b = false;  
console.log(a && b); // false (AND)  
console.log(a || b); // true (OR)  
console.log(!a); // false (NOT)
```

❏ **Tip:** Always prefer `===` (strict equality) over `==` in test scripts to avoid type coercion surprises.

Conditional Statements

Conditionals allow your script to make decisions based on the value of a variable or the result of an expression. They are fundamental to writing assertions and branching logic in test automation.

if / else

```
let statusCode = 404;

if (statusCode === 200) {
  console.log("Test Passed");
} else if (statusCode === 404) {
  console.log("Page Not Found");
} else {
  console.log("Unexpected Status");
}
```

switch

```
let browser = "Chrome";

switch (browser) {
  case "Chrome":
    console.log("Using Chrome");
    break;
  case "Firefox":
    console.log("Using Firefox");
    break;
  default:
    console.log("Unknown browser");
}
```

Use `switch` when comparing a single variable against multiple fixed values. It is cleaner than long `if-else` chains.

Loops: for and while

Loops allow you to execute a block of code repeatedly. In test automation, loops are commonly used to iterate over test data arrays, retry failed checks, or process lists of elements.

for Loop

Use when the number of iterations is known in advance.

```
let testCases = ["login","search","logout"];

for (let i = 0; i < testCases.length; i++) {
  console.log("Running: " + testCases[i]);
}
// Running: login
// Running: search
// Running: logout
```

while Loop

Use when iterations depend on a condition being true.

```
let retries = 0;

while (retries < 3) {
  console.log("Attempt: " + (retries + 1));
  retries++;
}
// Attempt: 1
// Attempt: 2
// Attempt: 3
```



SECTION 3

Functions in JavaScript

What is a Function?

Definition

A function is a reusable, named block of code designed to perform a specific task. Functions help you avoid repeating the same code, keep scripts organised, and make test utilities modular and maintainable.

Functions are defined once and can be called (invoked) multiple times with different inputs (parameters), producing different outputs (return values).

Basic Function Syntax

```
// Defining a function
function greetUser(name) {
  let message = "Hello, " + name + "!";
  return message;
}

// Calling the function
let result = greetUser("Alice");
console.log(result);
// Output: Hello, Alice!
```

Arrow Functions

Arrow functions are a more concise syntax for writing functions, introduced in ES6. They are widely used in modern JavaScript and in test automation frameworks like Cypress and Playwright. Understanding them is essential for reading framework documentation.

Traditional Function

```
function add(a, b) {  
  return a + b;  
}  
console.log(add(3, 4)); // 7
```

Arrow Function Equivalent

```
// Standard arrow function  
const add = (a, b) => {  
  return a + b;  
};  
  
// Concise (implicit return)  
const add = (a, b) => a + b;  
  
console.log(add(3, 4)); // 7
```

📌 **Trainer Note:** Arrow functions do not have their own `this` binding. This matters in event handlers but is less relevant for straightforward test utilities.

Scope: Global vs Local

Scope determines where a variable is accessible in your code. Understanding scope prevents naming conflicts and unexpected overwrites — a common source of bugs in test scripts.



```
let environment = "staging"; // Global scope
```

```
function runTest() {  
  let testName = "Login Test"; // Local scope  
  console.log(environment); // OK — accessible  
  console.log(testName); // OK — accessible here  
}
```

```
// console.log(testName); // ERROR — not accessible outside function
```

Hands-On Exercise 1: Write a Sum Function

Objective

Write a JavaScript function that accepts two numbers as parameters, returns their sum, and prints the result to the console. Practice using both regular and arrow function syntax.

Starter Code

```
// Task 1: Traditional function
function sumNumbers(a, b) {
  // Your code here
}

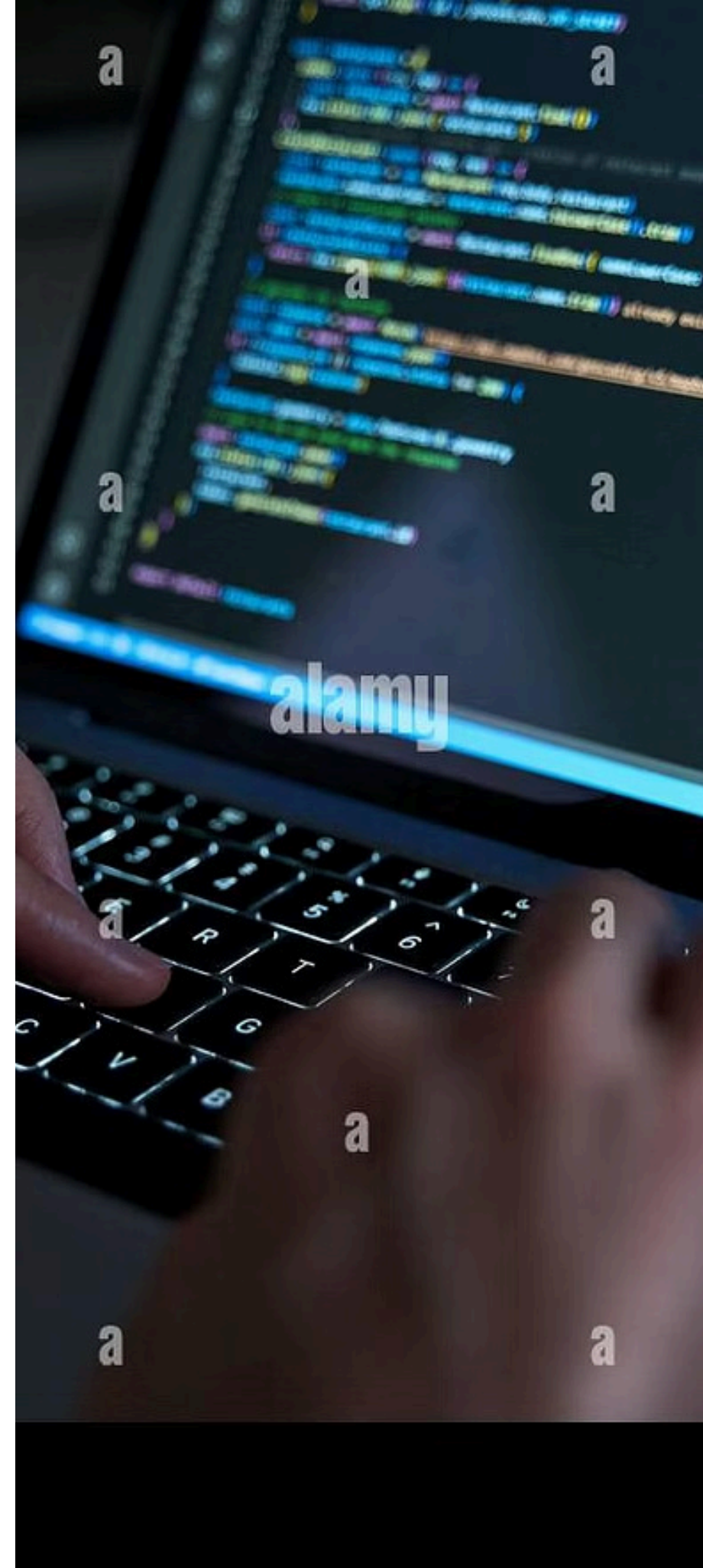
// Task 2: Arrow function version
const sumArrow = (a, b) => {
  // Your code here
};

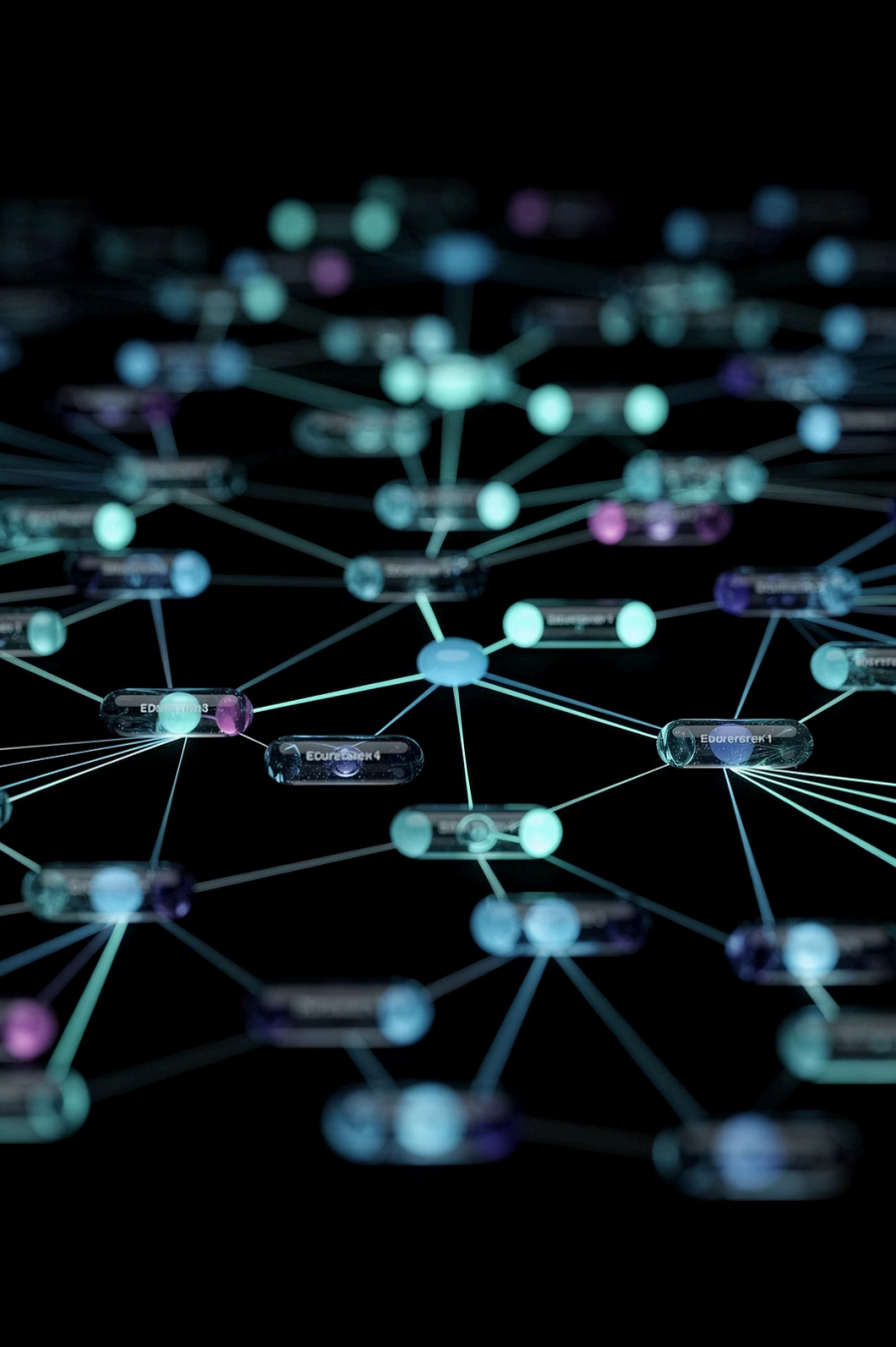
// Test your functions
console.log(sumNumbers(5, 10)); // Expected: 15
console.log(sumArrow(7, 3));    // Expected: 10
```

Expected Output

```
15
10
```

Bonus: Modify the function to also print: "The sum of 5 and 10 is 15"





SECTION 4

The Document Object Model (DOM)

What is the DOM?

The Definition

The Document Object Model (DOM) is a programming interface provided by the browser. When a browser loads an HTML page, it creates a structured representation of the document as a **tree of objects** — each HTML element becomes a node in this tree.

JavaScript can read and modify this tree in real time, enabling dynamic changes to content, structure, and styles without reloading the page.

Why DOM Matters for QA

→ Element Inspection

Understand how automation tools like Playwright and Selenium locate elements on the page

→ Script Injection

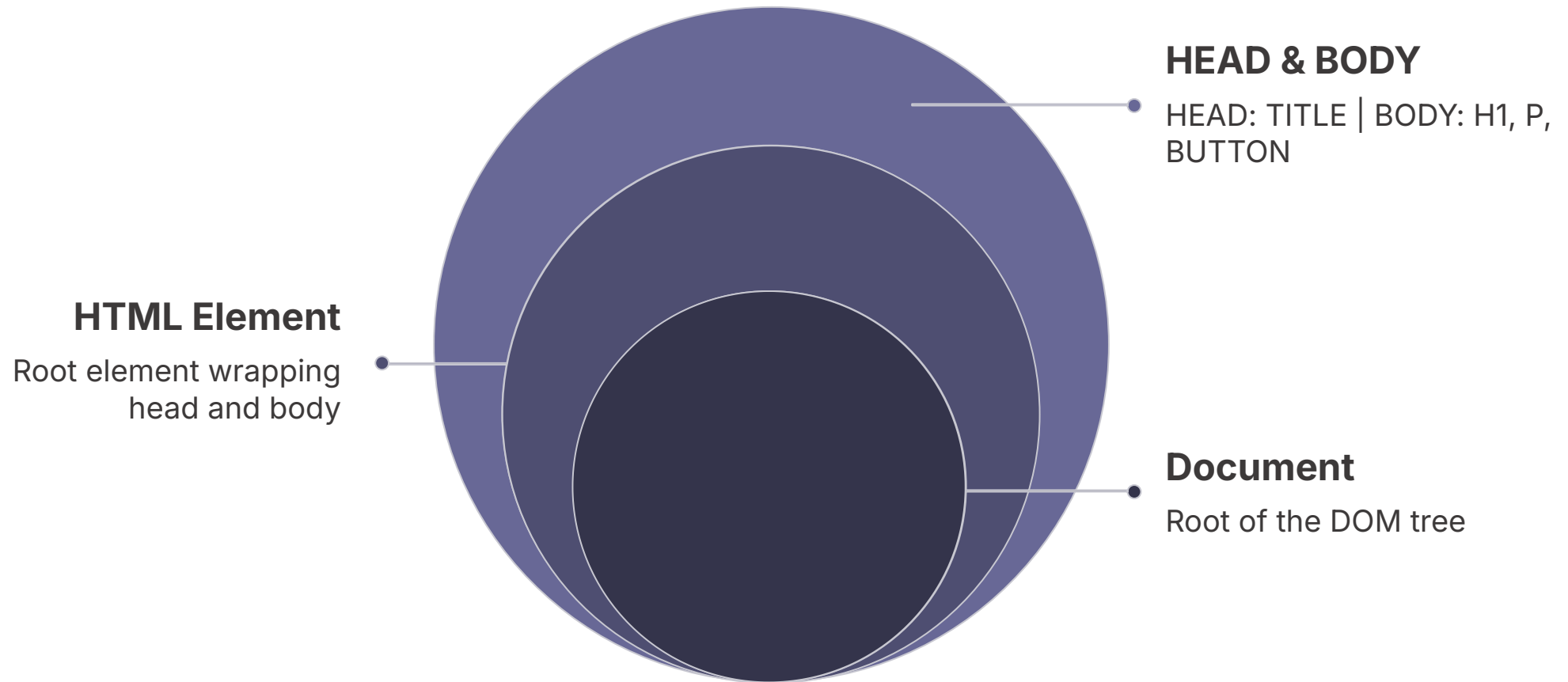
Execute JavaScript directly within automated tests using `executeScript()` or `evaluate()`

→ Dynamic Element Handling

Handle elements that appear or change after page load — a common challenge in modern SPAs

The DOM Tree Structure

Every HTML document is represented as a hierarchical tree. The browser builds this tree when parsing HTML. Understanding this hierarchy is fundamental to locating and interacting with elements in automation.



Each node in the tree is an **object** with properties (like `textContent`, `id`, `className`) and methods (like `click()`, `setAttribute()`). JavaScript navigates this tree using the global `document` object.

Accessing DOM Elements

Before you can manipulate an element, you must first retrieve a reference to it. JavaScript provides several built-in methods on the `document` object for this purpose.

`getElementById()`

Returns a single element with the specified id. Fast and precise — use when the element has a unique ID.

```
const btn =  
document.getElementById("submitBtn");
```

`querySelector()`

Returns the **first** element matching a CSS selector. Very flexible — supports IDs, classes, attributes, and pseudo-selectors.

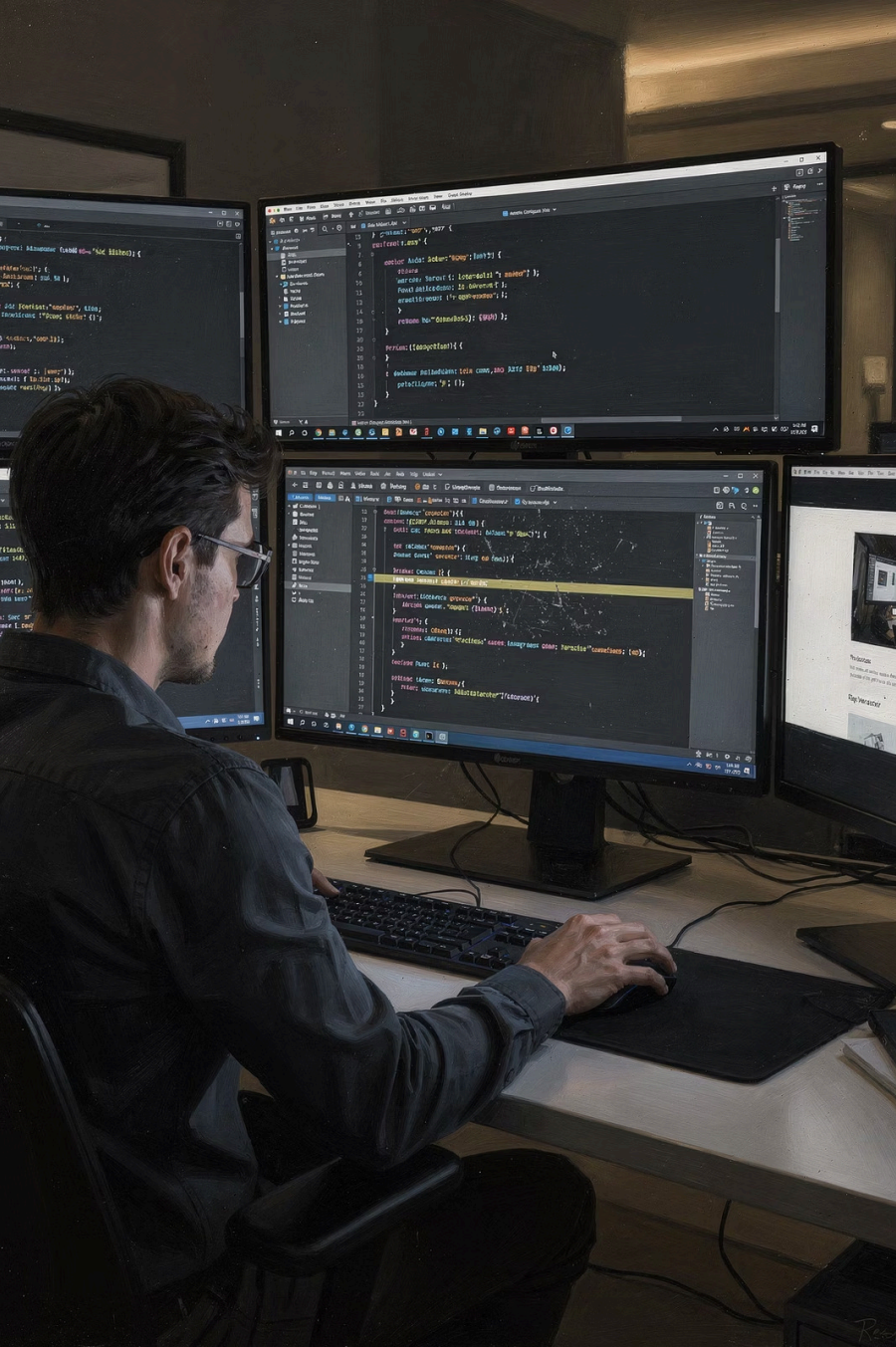
```
const el =  
document.querySelector(".error-msg");
```

`getElementsByClassName()`

Returns an **HTMLCollection** (live list) of all elements with the given class name. Iterate with a loop.

```
const items =  
document.getElementsByClassName("list-item");
```

📌 **Trainer Note:** `querySelector()` is the most versatile and is used extensively in Playwright's `page.locator()` and Cypress's `cy.get()`. Learners should get comfortable with CSS selector syntax.



SECTION 5

DOM Manipulation

Modifying Content, Attributes & Styles

Once you have a reference to a DOM element, you can change its content, HTML attributes, and visual styling entirely from JavaScript. These are the core operations used in browser automation scripts.

Text & HTML Content

```
const heading = document.getElementById("title");

// Change visible text only
heading.textContent = "Welcome Back!";

// Change inner HTML (renders tags)
heading.innerHTML = "<strong>Welcome!</strong>";
```

Attributes

```
const link = document.querySelector("a");
link.setAttribute("href", "https://newsite.com");
link.getAttribute("href"); // read it back
```

Inline Styles

```
const btn = document.getElementById("myBtn");

// Change individual style properties
btn.style.backgroundColor = "#1B1B27";
btn.style.color = "white";
btn.style.fontSize = "16px";
btn.style.display = "none"; // hide element
```

CSS Classes

```
const card = document.querySelector(".card");
card.classList.add("highlighted");
card.classList.remove("hidden");
card.classList.toggle("active");
```


Creating and Removing Elements

Creating New Elements

Use `document.createElement()` to build new DOM nodes and `appendChild()` or `insertBefore()` to place them in the tree.

```
// Create a new paragraph element
const newPara = document.createElement("p");
newPara.textContent = "Test result: PASSED";
newPara.style.color = "green";

// Append it to an existing container
const container = document.getElementById("results");
container.appendChild(newPara);
```

Removing Elements

Elements can be removed from the DOM using `remove()` on the element itself, or `removeChild()` from the parent.

```
// Method 1: Direct remove (modern)
const banner = document.getElementById("adBanner");
banner.remove();

// Method 2: Remove via parent
const parent = document.getElementById("wrapper");
const child = document.getElementById("popup");
parent.removeChild(child);
```

📄 **QA Use Case:** Injecting result markers into the page during visual regression checks or removing overlapping elements before screenshot capture.

Hands-On Exercise 2: DOM Manipulation Script

Objective

Write a JavaScript script that modifies page content using DOM manipulation methods. Open your browser's DevTools console (F12) to run the code on any live webpage.

Tasks

```
// Task 1: Change the page title
(browser tab)
document.title = "My Automated
Test Page";

// Task 2: Update the first
paragraph's text
const para =
document.querySelector("p");
if (para) para.textContent =
"Updated by JavaScript!";

// Task 3: Change button colour on
click
const btn =
document.querySelector("button");
if (btn) {
  btn.addEventListener("click",
function() {
  btn.style.backgroundColor =
"#1B1B27";
  btn.style.color = "white";
});
}
```

Verification Steps

- Confirm the browser tab title changes immediately
- Verify the paragraph text updates on the page
- Click the button and confirm the colour change



SECTION 6

Event Handling in JavaScript

What Are Events?

Events are signals fired by the browser in response to user interactions or changes in the page. JavaScript can listen for these signals and execute code in response — this is the foundation of event-driven programming.



click

Fires when a user clicks on an element. Most common event in UI automation for button and link interactions.



mouseover

Fires when the cursor moves onto an element. Used for testing hover states and tooltip visibility.



submit

Fires when a form is submitted. Used to validate form data before it is sent to the server.



keypress

Fires when a keyboard key is pressed. Useful for testing keyboard navigation and shortcut handling.

addEventListener()

The modern, recommended way to attach event listeners in JavaScript is the `addEventListener()` method. Unlike inline event attributes (e.g. `onclick="..."`), it allows you to attach multiple listeners to the same element without overwriting existing ones.

Syntax

```
element.addEventListener(eventType, handlerFunction);
```

Button Click Example

```
const btn = document.getElementById('testBtn');

btn.addEventListener('click', function() {
  const heading = document.getElementById('pageTitle');
  heading.textContent = 'Button was clicked!';
  heading.style.color = '#4CAF50';
  console.log('Click event fired successfully');
});
```

Form Submit Example

```
const form = document.getElementById('loginForm');

form.addEventListener('submit', function(event) {
  event.preventDefault(); // Stop page reload

  const username = document.getElementById('user').value;

  if (username === '') {
    alert('Username cannot be empty');
  } else {
    console.log('Form submitted for: ' + username);
  }
});
```

❏ **QA Note:** `event.preventDefault()` is critical in test scripts to prevent form submissions from navigating away from the page mid-test.

Hands-On Exercise 3: Event-Driven Page

Objective

Create an HTML page with a button that responds to a click event by displaying an alert and changing the page's background colour. This simulates real event-driven interactions tested in UI automation.

HTML & JavaScript

```
<!DOCTYPE html>
<html>
<body id="mainBody">
  <h1>Event Handling Exercise</h1>
  <button id="colorBtn">Click Me!</button>

  <script>
    const btn = document.getElementById("colorBtn");
    const body = document.getElementById("mainBody");

    btn.addEventListener("click", function() {
      alert("Button clicked! Changing background...");
      body.style.backgroundColor = "#1B1B27";
      body.style.color = "white";
      btn.textContent = "Clicked!";
    });
  </script>
</body>
</html>
```

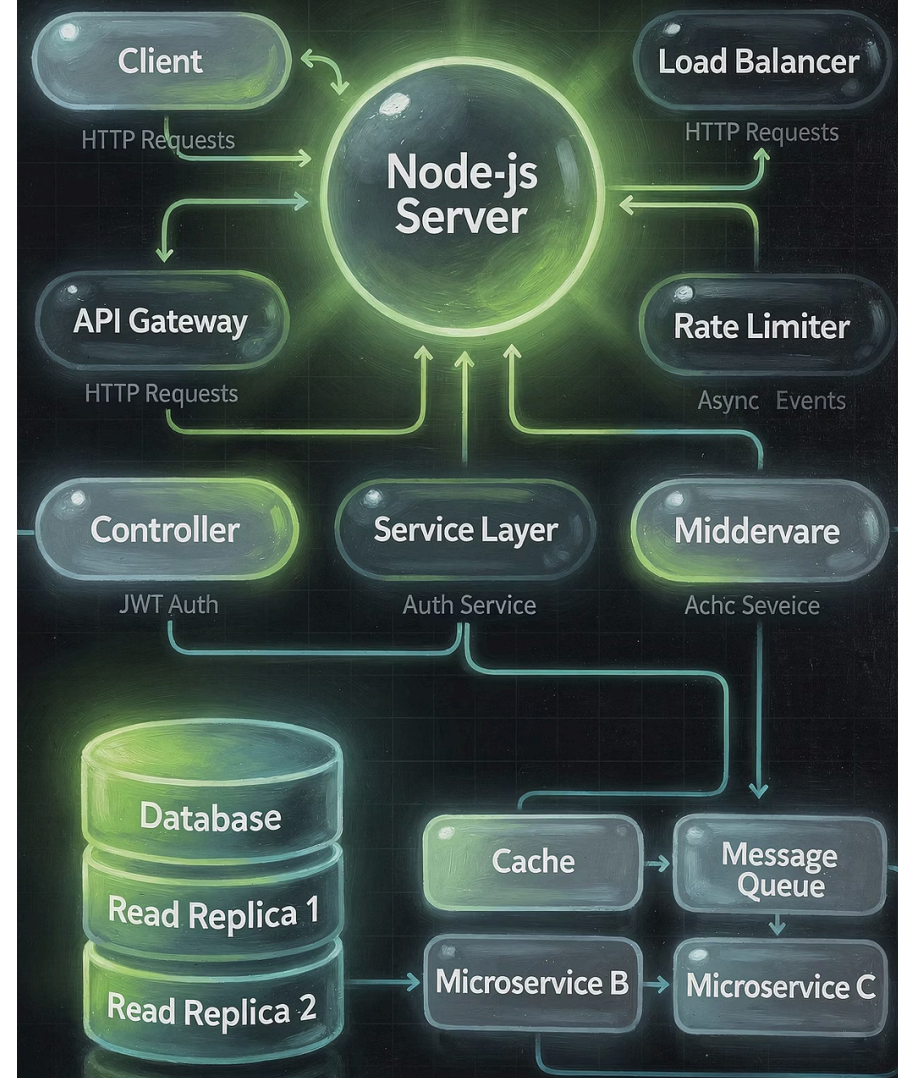
Verification

- Save as `exercise3.html` and open in a browser
- Click the button — confirm the alert appears
- Dismiss the alert — confirm the background and text colour change
- Confirm the button label updates to "Clicked!"

Interactions:
Clicking and Handling
Events in HTML

SECTION 7

Introduction to Node.js



What is Node.js?

The Runtime Environment

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It allows JavaScript to execute **outside of the browser** — on a server, local machine, or CI/CD pipeline.

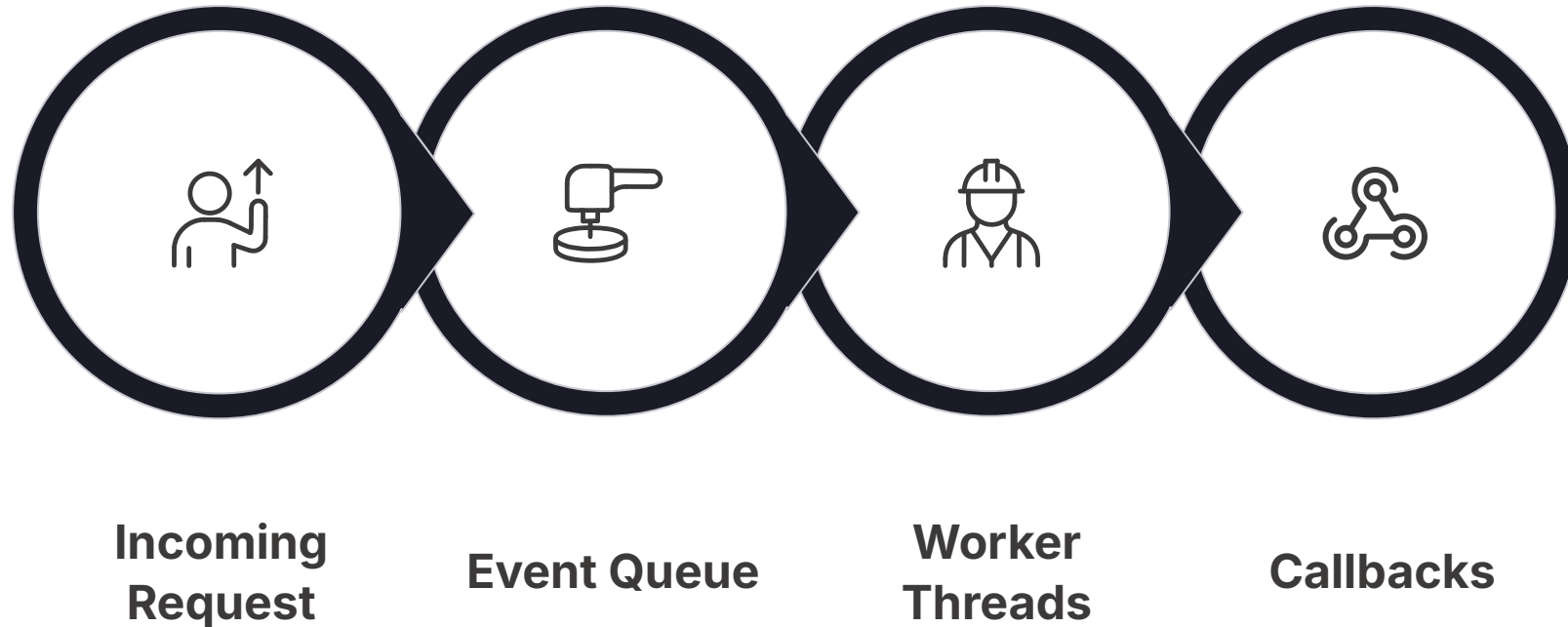
Released in 2009, Node.js transformed JavaScript from a front-end-only language into a full-stack, general-purpose scripting tool.

Node.js vs Browser JavaScript

Browser JS	Node.js
Runs in the browser sandbox	Runs on the local OS or server
Access to DOM, window, document	No DOM — no browser APIs
Limited file system access	Full file system & OS access
Used for UI interactions	Used for scripts & automation utilities
Sandboxed environment	Full system permissions

Node.js Architecture

Node.js uses an **event-driven, non-blocking I/O model**. Unlike traditional server languages that create a new thread per request, Node.js processes everything in a single thread using an event loop — making it lightweight and efficient for I/O-heavy tasks like file reading and API calls.



This architecture makes Node.js ideal for running test automation utilities, file processing scripts, and API interaction tools — all common tasks in a QA engineer's daily workflow.

SECTION 8

Node.js Basics

```
1 // ...
2 {itds tðey toan paont icaots aoe sto sont ieenpotli iedo ==} |
3 @aâ6}
4 I loabt ceûgact server godittc tieze (desids) somier ot ?000:?)
5 "thlt dines younue: {
6   "3 0e ioettt thie: {
7     "tâdestedlll/ snitgcsfs Ieoits fifty loather oderat@ne: '{
8     steneriated sorvesttûs"ton0;
9     'Guatus anp epiuney i=- nilest thall;
10    %I feat snn cun doso: {
11      npm sciide jdse:
12      fruseft ass vites thas dasson: {
13
14        noode -v {
15          ppm install
16          npm ieffes iahct: $ 186 -->3 ("00000)
17          ppm iescanchens 0f"0oi2}
18          npm run dev : " "(lli afanunstall")
19          nuld completed suc.vessfull;
20          node server.js
21        }
22      }
23      Server running on http:// localhost: 30000:
24      node server.js
25      Buil/ completed successfully:
26
27      {
28        nodlen running eor. htt:p://locaahnost:3 00.0
29      }
30
31  }
32
33
34 }
```

Installing Node.js & Running Your First Script

01

Download & Install

Visit **nodejs.org** and download the LTS (Long-Term Support) version for your operating system. The installer includes both Node.js and npm (Node Package Manager).

02

Verify Installation

```
node --version # e.g. v20.11.0
npm --version  # e.g. 10.2.4
```

03

Write a Script

```
// hello.js
console.log("Hello from Node.js!");
console.log("Current time: " + new Date().toLocaleTimeString());
```

04

Execute It

```
node hello.js
# Output:
# Hello from Node.js!
# Current time: 14:35:22
```

 **Trainer Note:** Have learners run these commands live in the terminal. Confirm all machines have Node.js v18+ installed before the session starts.

Hands-On Exercise 4: Node.js Console Script

Objective

Create a Node.js script that prints the current date, simulates receiving user input, and displays a formatted output message. This mirrors common utility script patterns used in QA pipelines.

Script: dateScript.js

```
// dateScript.js

// Task 1: Print current date and time
const now = new Date();
console.log("Current Date: " +
now.toLocaleDateString());
console.log("Current Time: " +
now.toLocaleTimeString());

// Task 2: Simulate user input using process.argv
const userName = process.argv[2]
|| "QA Engineer";

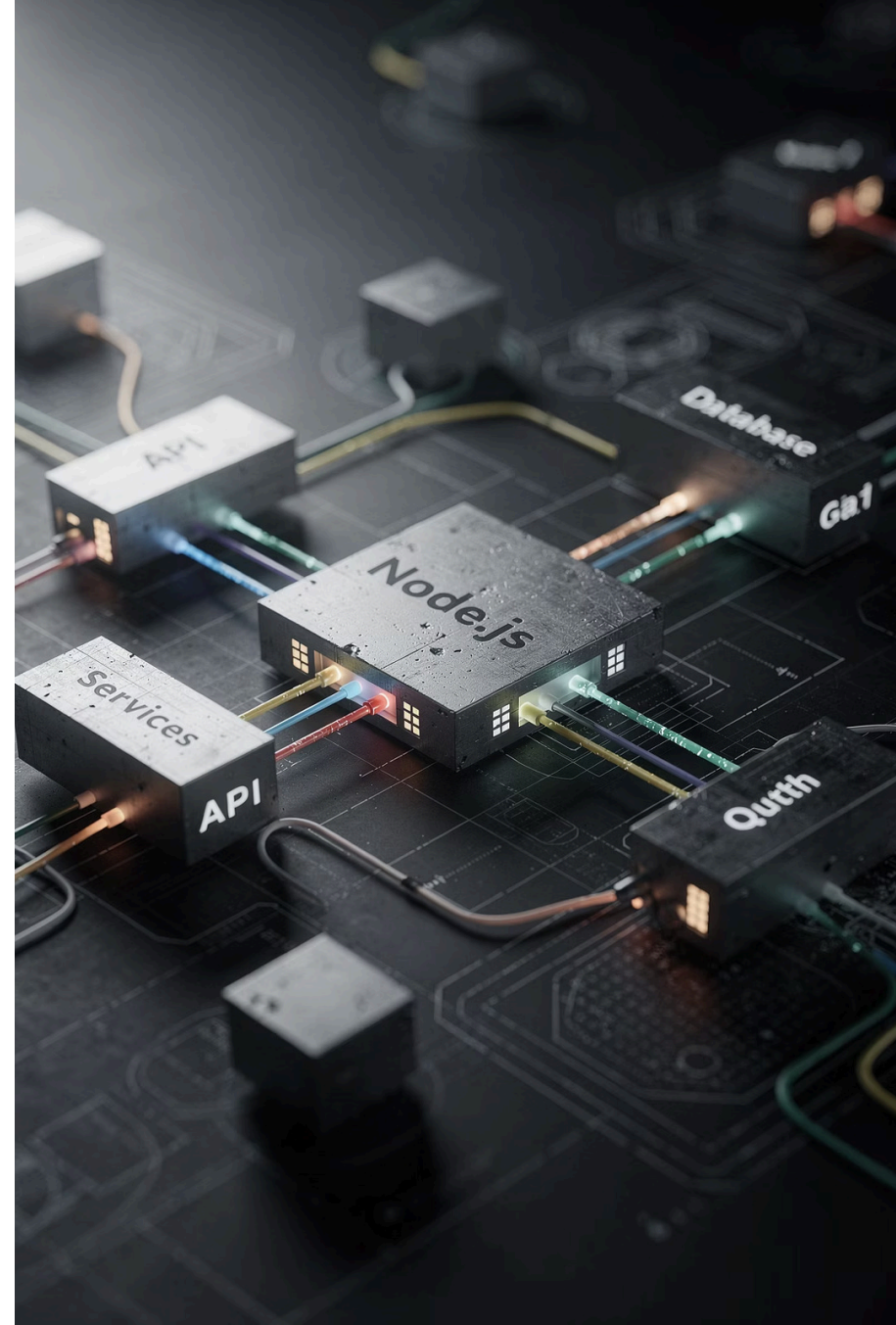
// Task 3: Display formatted message
console.log("-----");
console.log("Welcome, " +
userName + "!");
console.log("Test session started at:
" + now.toISOString());
console.log("-----");
```

Run & Verify

```
node dateScript.js Alice
# Output:
# Current Date: 15/01/2025
# Current Time: 14:35:22
# -----
# Welcome, Alice!
# Test session started at: 2025-01-
15T14:35:22.000Z
# -----
```

SECTION 9

Node.js Modules



What Are Modules?

A module is a self-contained JavaScript file that encapsulates related functionality. Node.js uses a module system to organise code into reusable, shareable units — preventing pollution of the global scope and enabling code to be split across multiple files.

Exporting from a Module

```
// mathUtils.js
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }

// Export functions for use elsewhere
module.exports = { add, subtract };
```

Importing a Module

```
// main.js
const math = require('./mathUtils');

console.log(math.add(10, 5)); // 15
console.log(math.subtract(10, 5)); // 5
```

Built-In Node.js Modules

→ fs (File System)

Read, write, and manage files on the local system — useful for test data management and log parsing

→ http

Create HTTP servers or make HTTP requests — useful for lightweight API testing utilities

→ path

Resolve and manipulate file system paths in a cross-platform way — essential for CI scripts

Reading a File with the fs Module

The built-in `fs` (File System) module is one of the most commonly used Node.js modules in QA contexts. It enables scripts to read test data files, parse configuration, and write test output — all without any additional installation.

testData.txt

```
username=testuser  
password=pass123  
environment=staging
```

Asynchronous Read (Recommended)

```
// readFile.js  
const fs = require("fs");  
  
fs.readFile("testData.txt", "utf8", (err, data) => {  
  if (err) {  
    console.error("Error reading file:", err.message);  
    return;  
  }  
  console.log("File contents:");  
  console.log(data);  
});
```

Synchronous Read (Simpler for Scripts)

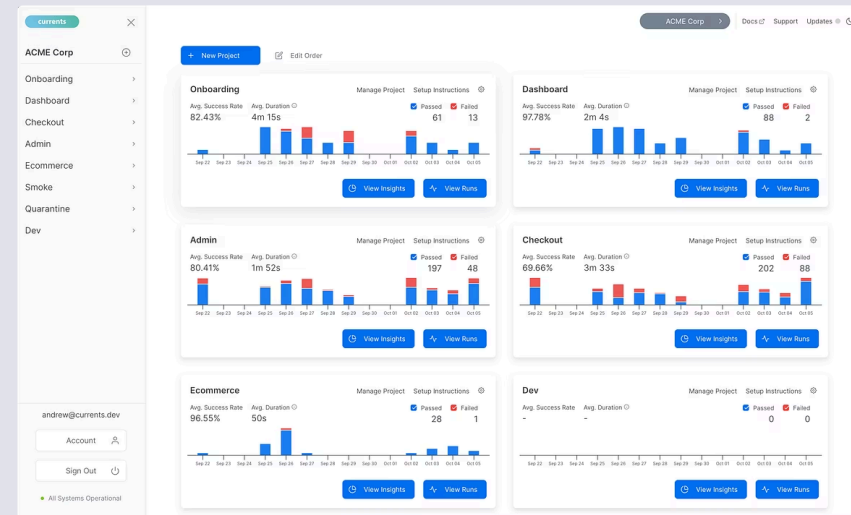
```
// readFileSync.js  
const fs = require("fs");  
  
try {  
  const data = fs.readFileSync("testData.txt", "utf8");  
  console.log("File contents:");  
  console.log(data);  
} catch (err) {  
  console.error("Could not read file:", err.message);  
}
```

Writing to a File

```
const fs = require("fs");  
  
const log = "Test run completed at " + new Date().toISOString();  
fs.writeFileSync("testLog.txt", log);  
console.log("Log written successfully.");
```

SECTION 10

Node.js for Testing & Automation



Node.js in the Automation Ecosystem

Node.js serves as the runtime foundation for virtually every major modern test automation framework. Understanding Node.js enables you to configure, extend, and debug these frameworks with confidence rather than treating them as black boxes.



Playwright

Microsoft's end-to-end testing framework. Supports Chromium, Firefox, and WebKit. Written in Node.js with powerful selector and assertion APIs.



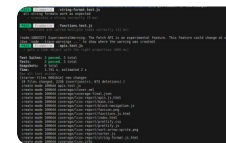
Cypress

An in-browser E2E testing tool with a real-time test runner. Built entirely on Node.js with a developer-friendly API and built-in assertions.



API Testing

Tools like Axios and Supertest allow Node.js scripts to send HTTP requests, validate responses, and integrate API checks into CI/CD pipelines.



Jest

Facebook's JavaScript testing framework for unit and integration tests. Ideal for testing utility functions and API response validators.

Utility Scripts in Node.js for QA

Beyond test frameworks, Node.js excels at automation utility scripts — small, focused programs that handle repetitive QA tasks. These scripts can be integrated into CI pipelines and scheduled jobs.

Test Data Generator

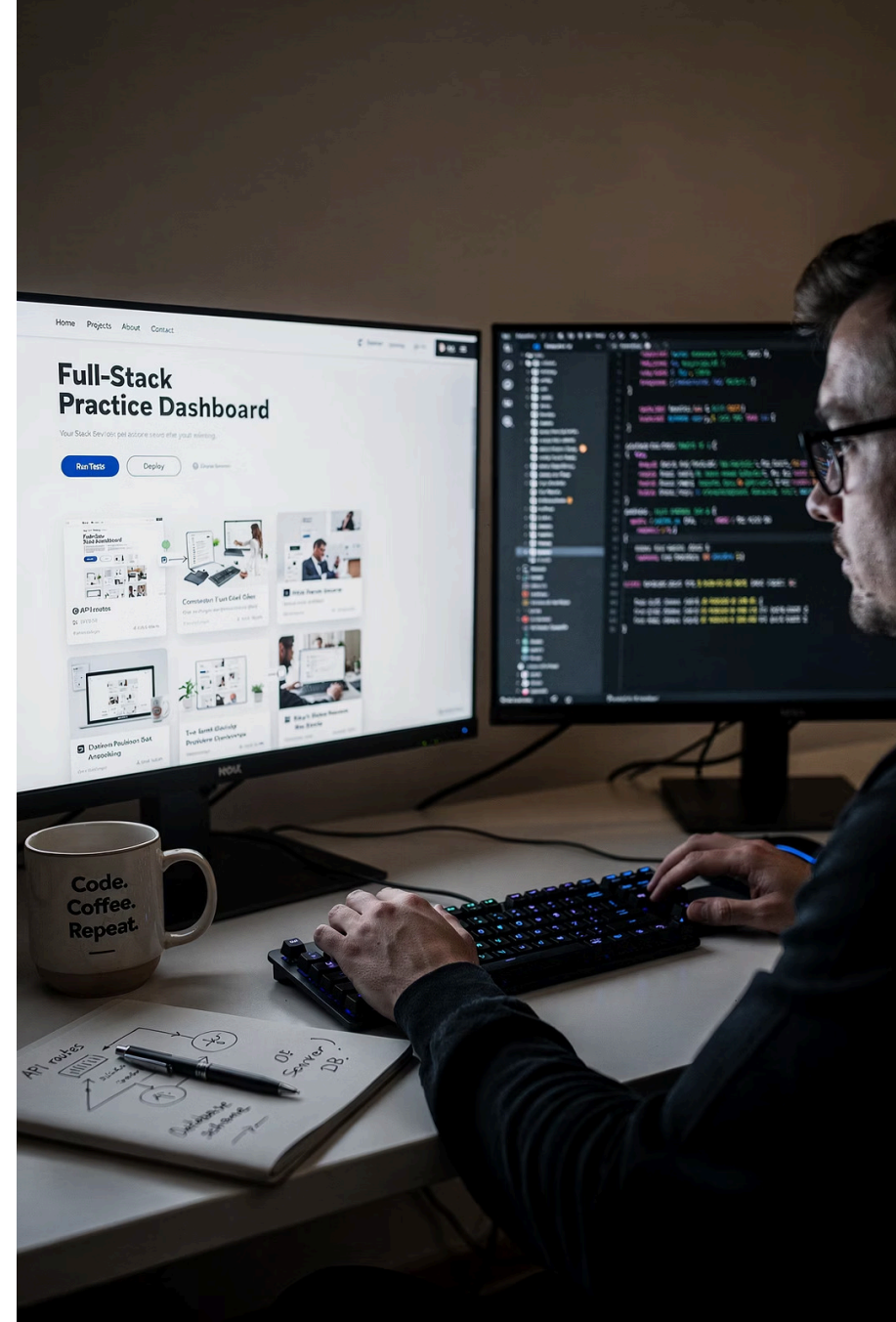
```
const users = [];  
for (let i = 1; i <= 10; i++) {  
  users.push({  
    id: i,  
    username: "testuser" + i,  
    email: "user" + i + "@test.com"  
  });  
}  
console.log(JSON.stringify(users, null, 2));
```

API Health Check Script

```
const http = require("http");  
  
http.get("http://localhost:3000/health", (res) => {  
  console.log("Status:", res.statusCode);  
  if (res.statusCode === 200) {  
    console.log("API is healthy ✓");  
  } else {  
    console.log("API check failed ");  
  }  
});
```

SECTION 11

End-to-End Mini Exercise



Mini Project: Interactive Webpage with Node.js

This capstone exercise brings together everything covered in this training. You will build a small interactive webpage, apply DOM manipulation and event handling, and execute a companion Node.js script to validate the scenario from the command line.

1

Create the HTML Page

Build `index.html` with a heading, paragraph, input field, and a button. Include a `<script>` block to handle events and DOM updates.

2

Add JavaScript Interactions

When the button is clicked: read the input value, update the paragraph text with a personalised message, and change the button's background colour.

3

Write a Node.js Companion Script

Create `runTest.js` that logs a test session header, simulates the user interaction scenario, and prints a confirmation message using `console.log()`.

4

Run & Verify

Open the HTML page in a browser and test the interactions. Then run `node runTest.js` in the terminal and verify the expected console output.

Mini Exercise: Solution Reference

index.html (key script block)

```
<body>
  <h1 id='greeting'>Welcome!</h1>
  <p id='message'>Enter your name below.</p>
  <input id='nameInput' type='text' placeholder='Your name' />
  <button id='greetBtn'>Greet Me</button>

  <script>
    document.getElementById('greetBtn')
      .addEventListener('click', function() {
        const name =
document.getElementById('nameInput').value;
        document.getElementById('message').textContent =
          "Hello, " + name + "! Welcome to JS Training.";
        this.style.backgroundColor = "#1B1B27";
        this.style.color = "white";
      });
  </script>
</body>
```

runTest.js

```
// runTest.js — Node.js companion script
const fs = require("fs");

console.log("=====");
console.log("JS Training — Mini Exercise Run");
console.log("=====");

const timestamp = new Date().toISOString();
console.log("Test session started: " + timestamp);
console.log("");
console.log("Scenario: User enters name and clicks Greet");
console.log("Expected: Paragraph updates with greeting");
console.log("Expected: Button changes to dark colour");
console.log("");

const log = "Session: " + timestamp + " | Status: COMPLETE\n";
fs.writeFileSync("testSession.log", log);
console.log("Session log written to testSession.log ✓");
console.log("=====");
```

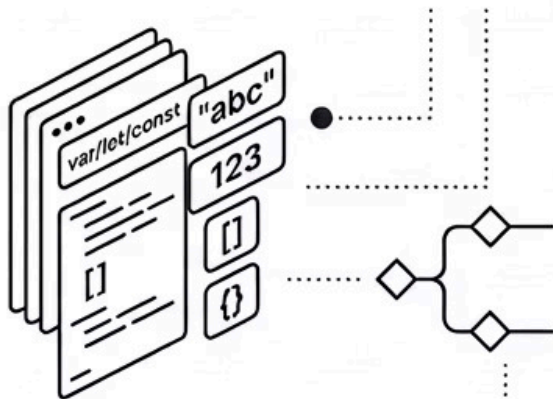



SUMMARY

Course Summary

What We Covered: Key Concepts Recap

JavaScript Fundamentals



Data Types & Variables
Loops & Conditionals
Functions & Scope

DOM Manipulation



Node.js

JavaScript outside browser

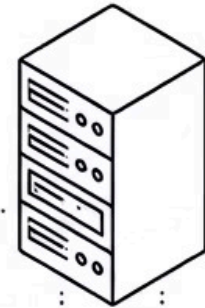
Event-browser

JavaScript outside

Built-in Modules

Event-driven architecture

Automation



JavaScript Fundamentals: Quick Reference

Variables

Use `const` by default, `let` when reassignment is needed, avoid `var`

Data Types

String, Number, Boolean, Array, Object — JavaScript is dynamically typed

Conditionals

Use `if/else` for flexible conditions; `switch` for fixed value comparisons

Loops

`for` when iterations are known; `while` when dependent on a condition

Functions

Use named functions for clarity; arrow functions for concise callbacks

Strict Equality

Always use `===` over `==` to avoid type coercion bugs in assertions

Common Interview Questions

These questions appear frequently in QA Engineer, SDET, and Automation Tester interviews. Review them thoroughly and ensure you can answer with practical code examples.

What is the difference between `var`, `let`, and `const`?

Focus on scope (function vs block), hoisting behaviour, and mutability rules.

What is the DOM and how do you access elements?

Explain the tree structure and demonstrate `getElementById`, `querySelector`, and traversal methods.

What is Node.js and how does it differ from browser JavaScript?

Highlight the V8 runtime, event-driven model, absence of DOM, and access to the file system and OS.

How does `addEventListener` work and why is it preferred?

Explain event types, the callback pattern, and the ability to attach multiple listeners without overwriting.

What are Node.js modules and how do you use them?

Explain `require()` for importing, `module.exports` for exporting, and name key built-in modules.

MCQ Revision Round

Test your understanding of the full course content with these revision questions. These are structured similarly to certification and interview MCQ assessments.

Q1: Variable Declaration

Which keyword should you use when a variable's value will never change after declaration?

A. var **B.** let **C.** const **D.** static

✓ **Answer: C — const**

Q2: DOM Access

Which method returns the first element matching a CSS selector?

A. getElementById() **B.** querySelector() **C.** getElementsByTagName() **D.** findElement()

✓ **Answer: B — querySelector()**

Q3: Node.js Modules

Which built-in Node.js module is used to read and write files from the local file system?

A. http **B.** path **C.** fs **D.** os

✓ **Answer: C — fs (File System)**

Q4: Event Handling

What does calling `event.preventDefault()` inside a form's submit handler achieve?

A. Removes all event listeners **B.** Stops the page from reloading on form submission **C.** Disables the submit button **D.** Clears all form fields

✓ **Answer: B**

Next Steps in Your Learning Journey



This Course — Completed

JavaScript fundamentals, DOM manipulation, event handling, Node.js basics and modules



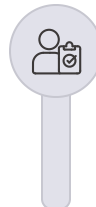
Advanced JavaScript

Promises, async/await, error handling, closures, and ES6+ features like destructuring and spread operators



Test Automation Frameworks

Deep dive into Playwright or Cypress — page objects, fixtures, test configuration, and CI/CD integration



API Testing with Node.js

REST API testing using Axios and Supertest, response validation, authentication handling, and test data management



Recommended Learning Resources

Documentation & References

→ **MDN Web Docs**

developer.mozilla.org —
the definitive JavaScript
and DOM reference.
Bookmark it.

→ **Node.js Official Docs**

nodejs.org/en/docs —
module API reference and
getting started guides

→ **Playwright Docs**

playwright.dev —
comprehensive API guide
and test examples

→ **Cypress Docs**

docs.cypress.io —
interactive guides with
live examples

Practice Platforms

→ **freeCodeCamp**

Free interactive
JavaScript curriculum
covering fundamentals
through advanced topics

→ **JavaScript.info**

In-depth, beginner-
friendly tutorials covering
every JavaScript concept
with exercises

→ **The Odin Project**

Full-stack curriculum with
hands-on JavaScript
projects including DOM
and Node.js

→ **Exercism.io**

Mentored code exercises
in JavaScript — ideal for
improving scripting
fluency

Thank You

Keep scripting. Keep testing.

You have completed **JavaScript Fundamentals, DOM Manipulation & Introduction to Node.js**. You now have the foundational scripting skills to contribute meaningfully to any modern test automation team. Apply these concepts consistently, practise with real projects, and continue building upward.

JAVASCRIPT

DOM

NODE.JS

QA AUTOMATION

SDET

